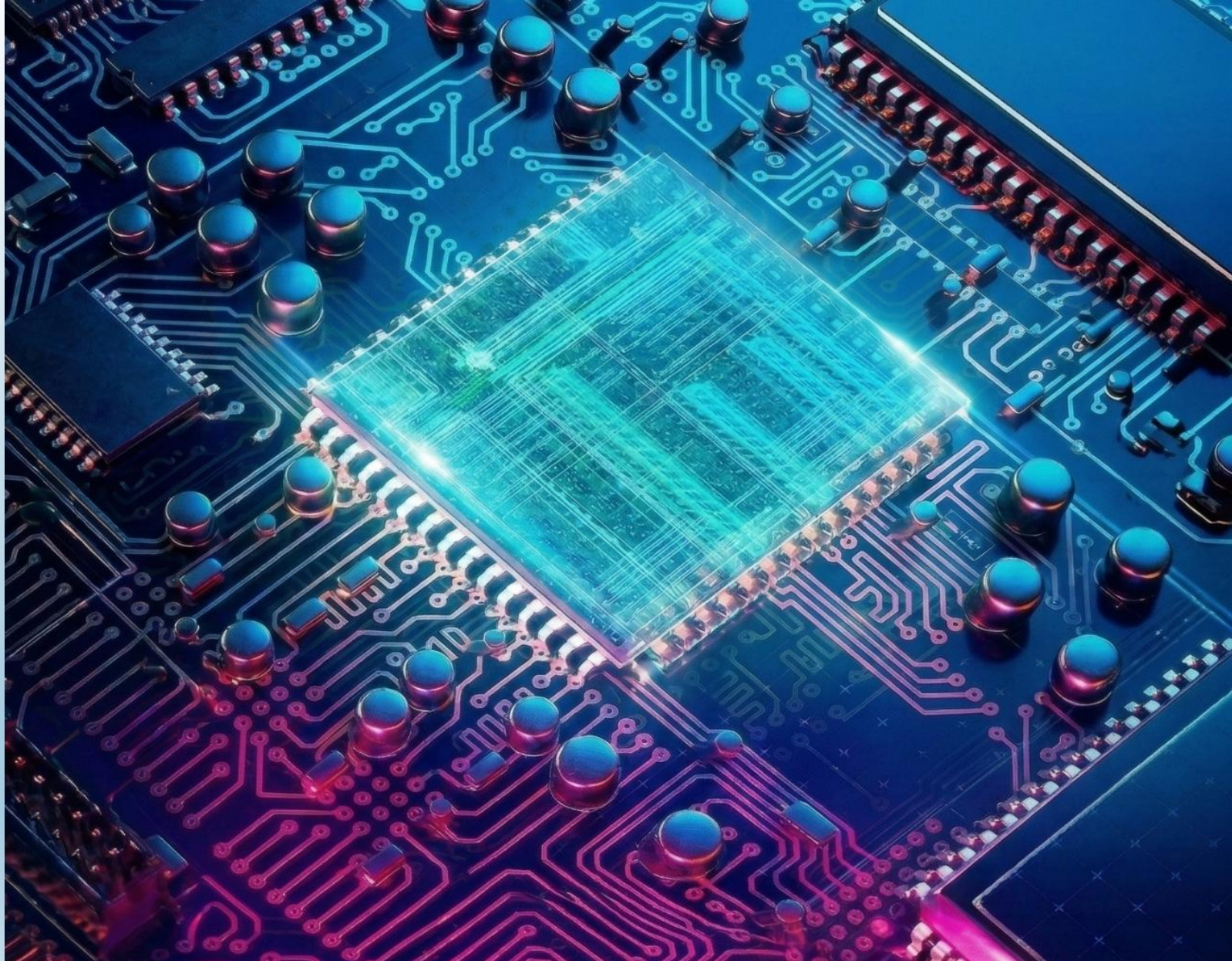




Digital Verification Assignment 3

Name: Mohamed Torki

Alexandria University, Faculty of Engineering
Electronics and Communications Department

Question 1:

Testbench:

```
Assignments > A3 > Q1 > tb.sv > tb
1  import testing_pkg::*;
2  module tb;
3      Transaction tr = new();
4      byte operand1, operand2;
5      bit clk, rst;
6      opcode_e opcode;
7      byte out;
8
9      integer correct_count, error_count;
10     byte expected;
11     initial begin
12         clk = 0;
13         forever begin
14             #1    clk = ~clk;
15             tr.clk = clk;
16         end
17     end
18     alu_seq DUT(
19         .clk(clk),
20         .rst(rst),
21         .operand1(operand1),
22         .operand2(operand2),
23         .out(out),
24         .opcode(opcode)
25     );
```



```

27     initial begin
28         correct_count = 0;
29         error_count = 0;
30         //alu_seq3
31         assert_reset();
32         //alu_seq4
33         repeat(32) begin
34             assert(tr.randomize);
35             operand1 = tr.operand1;
36             operand2 = tr.operand2;
37             opcode = tr.opcode;
38             calc_expected(expected);
39             check_result(expected);
40         end
41         assert_reset();
42         $display("%t: At end of test error count is %0d and correct count = %0d",
43             $time, error_count, correct_count);
44         $stop();
45     end

```

```

47     task calc_expected(output byte expected);
48         case (opcode)
49             ADD:    expected = operand1 + operand2;
50             SUB:    expected = operand1 - operand2;
51             MULT:   expected = operand1 * operand2;
52             DIV:    expected = operand1 / operand2;
53             default: expected = 0;
54         endcase
55     endtask

56
57     task check_result(input byte expected);
58         @(negedge clk)
59         if(expected != out)
60             begin
61                 $display("At Time %t: opcode=%s | operand1=%0d | operand2=%0d | out=%0d | expected=%0d",
62                     $time, opcode, operand1, operand2, out, expected);
63                 error_count++;
64             end
65         else
66             correct_count++;
67     endtask

```

```

69     task assert_reset;
70         rst = 1;
71         @(negedge clk);
72         check_result(0);
73         rst = 0;
74     endtask
75
76 endmodule

```



Package:

Labs > CW3 > EX2 > testing_pkg.sv > {} testing_pkg > Transaction

```

1  package testing_pkg;
2  typedef enum {ADD, SUB, MULT, DIV} opcode_e;
3  class Transaction;
4      rand opcode_e opcode;
5      rand byte operand1;
6      rand byte operand2;
7      bit clk;
8
9      constraint c{
10         opcode dist {ADD := 80, SUB := 80, [MULT:DIV] := 20};
11         operand1 dist {-128 := 80, 127 := 80, 0 := 5, [1:126] := 5, [-127:-1] := 5};
12     }
13

```

```

16  covergroup CovCode @(posedge clk);
17      //alu_seq0
18      opcode_cp: coverpoint opcode{
19         bins ADDorSUB = {ADD, SUB};
20         bins ADDthenSUB = (ADD => SUB);
21         illegal_bins noDIV = {DIV};
22     }
23      //alu_seq1
24      operand1_cp: coverpoint operand1{
25         bins max_neg = {-128};
26         bins zero = {0};
27         bins max_pos = {127};
28         bins others = default;
29     }
30      //alu_seq2
31      opcode_operand: cross opcode_cp, operand1_cp{
32         option.weight = 5;
33         option.cross_auto_bin_max = 0;
34         bins ADDorUB_max_pos = binsof(opcode_cp.ADDorSUB) && binsof(operand1_cp.max_pos);
35         bins ADDorUB_max_neg = binsof(opcode_cp.ADDorSUB) && binsof(operand1_cp.max_neg);
36     }
37  endgroup
38  function new();
39      CovCode = new();
40  endfunction
41  endclass
42  endpackage

```



Design:

Renamed Design File to `alu_seq.v` and Imported my package to the design

```
1  import testing_pkg::*;
2  module alu_seq(operand1, operand2, clk, rst, opcode, out);
3  input byte operand1, operand2;
4  input clk, rst;
5  input opcode_e opcode;
6  output byte out;
7
8  always @(posedge clk) begin
9      if (rst)
10         out <= 0;
11     else
12         case (opcode)
13             ADD: out <= operand1 + operand2;
14             SUB: out <= operand1 - operand2;
15             MULT: out <= operand1 * operand2;
16             DIV: out <= operand1 / operand2;
17             default: out <= 0;
18         endcase
19     end
20 endmodule
```



Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
alu_seq0	The ALU opcode must exercise ADD and SUB operations, including a direct ADD→SUB transition. DIV must never appear as it is an illegal operation.	Randomization of opcode from Transaction class. Constraints must bias toward ADD and SUB and must exclude DIV from ever being driven.	opcode_cp: bin ADDorSUB={ADD,SUB}, transition bin ADDthenSUB=(ADD=>SUB), illegal_bins noDIV={DIV} — sampled on posedge clk.	a checker checks if the output is correct.
alu_seq1	operand1 must cover its boundary values (max negative -128, zero, max positive 127) as well as all other values.	Randomization of operand1 from Transaction class, biased toward boundary values -128, 0, and 127 to ensure bin hits.	operand1_cp: bins max_neg={-128}, zero={0}, max_pos={127}, others=default — sampled on posedge clk.	a checker checks if the output is correct.
alu_seq2	The combination of ADD or SUB opcode with boundary operand1 values (max positive and max negative) must be explicitly exercised to cover arithmetic corner cases.	Randomization must jointly produce ADD/SUB opcodes alongside operand1=127 and operand1=-128 within the same stimulus cycle.	opcode_operand cross: bins ADDorSUB_max_pos = binsof(ADDorSUB) && binsof(max_pos), bins ADDorSUB_max_neg = binsof(ADDorSUB) && binsof(max_neg). Auto cross bins disabled (cross_auto_bin_max=0), cross weighted at 5.	a checker checks if the output is correct.
alu_seq3	When reset is asserted, the output of the ALU should be 0 regardless of operands or opcode.	Directed stimulus: rst forced to 1 via assert_reset task at the start of simulation before any randomization, then deasserted. Repeated again after the random loop.	-	a checker checks if the output is correct.
alu_seq4	The ALU must produce correct outputs for ADD, SUB, MULT, and DIV operations across 32 randomized stimulus iterations.	repeat(32) loop calling tr.randomize() each iteration to randomize operand1, operand2, and opcode from the Transaction class constraints.	-	a checker checks if the output is correct.

Do File:

```

Assignments > A3 > Q1 > ≡ run.do
1  vlib work
2  vlog testing_pkg.sv alu_seq.sv tb.sv +cover -covercells
3  vsim -voptargs=+acc work.tb -cover
4  add wave *
5  coverage save tb.ucdb -onexit
6  run -all
7  coverage exclude -du alu_seq -togglenode opcode
8  coverage exclude -src alu_seq.sv -line 17 -code b
9  coverage exclude -src alu_seq.sv -line 17 -code s
10 |

```



Code Coverage Report:

Branch Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	6	6	0	100.00%

Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	6	6	0	100.00%

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	52	52	0	100.00%

Functional Coverage Report:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/testing_pkg/Transaction		100.00%							
CVP CovCode		100.00%	100	100.00...		✓			auto(0)
CVP CovCode::opcode_cp		100.00%	100	100.00...		✓			
CVP CovCode::operand1_cp		100.00%	100	100.00...		✓			
CROSS CovCode::opcode_oper...		100.00%	100	100.00...		✓			
INST /testing_pkg::Transaction...		100.00%	100	100.00...		✓			0
CVP opcode_cp		100.00%	100	100.00...		✓			
illegal_bin noDIV		2	-	-		✓			
bin ADDorSUB		32	1	100.00...		✓			
bin ADDthenSUB		8	1	100.00...		✓			
CVP operand1_cp		100.00%	100	100.00...		✓			
bin max_neg		1	1	100.00...		✓			
bin zero		2	1	100.00...		✓			
bin max_pos		2	1	100.00...		✓			
default bin others		31	-	-		✓			
CROSS opcode_operand		100.00%	100	100.00...		✓			
bin ADDorUB_max_pos		2	1	100.00...		✓			
bin ADDorUB_max_neg		1	1	100.00...		✓			



```

=====
=== Instance: /testing_pkg
=== Design Unit: work.testing_pkg
=====

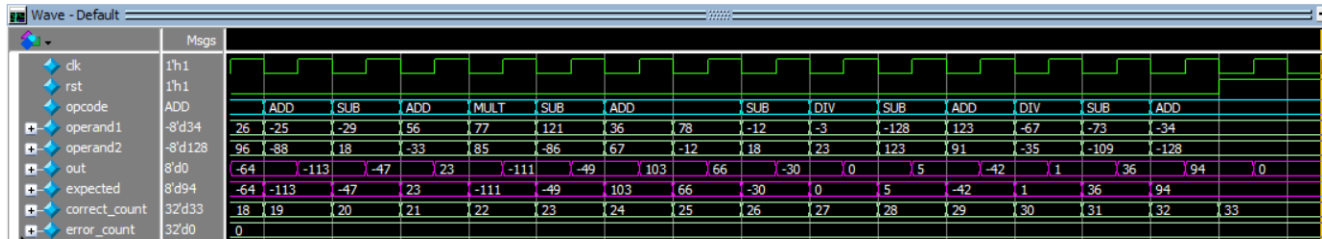
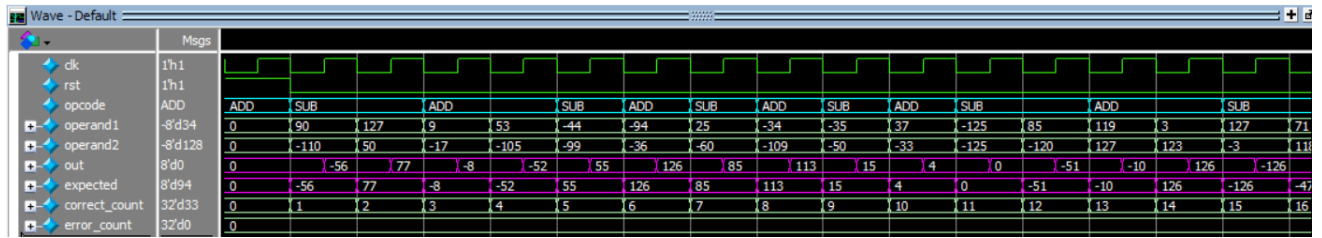
Covergroup Coverage:
  Covergroups          1      na      na    100.00%
  Coverpoints/Crosses  3      na      na      na
  Covergroup Bins      7      7      0    100.00%
-----
Covergroup              Metric      Goal      Bins      Status
-----
TYPE /testing_pkg/Transaction/CovCode    100.00%    100      -      Covered
covered/total bins:                      7         7      -
missing/total bins:                      0         7      -
% Hit:                                   100.00%    100      -
Coverpoint opcode_cp    100.00%    100      -      Covered
covered/total bins:      2         2      -
missing/total bins:      0         2      -
% Hit:                   100.00%    100      -
Coverpoint operand1_cp  100.00%    100      -      Covered
covered/total bins:      3         3      -
missing/total bins:      0         3      -
% Hit:                   100.00%    100      -
Cross opcode_operand    100.00%    100      -      Covered
covered/total bins:      2         2      -
missing/total bins:      0         2      -
% Hit:                   100.00%    100      -
Covergroup instance \testing_pkg::Transaction::CovCode
covered/total bins:      7         7      -      Covered
missing/total bins:      0         7      -
% Hit:                   100.00%    100      -
Coverpoint opcode_cp    100.00%    100      -      Covered
covered/total bins:      2         2      -
missing/total bins:      0         2      -
% Hit:                   100.00%    100      -
illegal_bin noDIV        2         -      -      Occurred
bin ADDorSUB             32        1      -      Covered
bin ADDthenSUB           8         1      -      Covered

Coverpoint operand1_cp    100.00%    100      -      Covered
covered/total bins:      3         3      -
missing/total bins:      0         3      -
% Hit:                   100.00%    100      -
bin max_neg              1         1      -      Covered
bin zero                  2         1      -      Covered
bin max_pos              2         1      -      Covered
default bin others       31        -      -      Occurred
Cross opcode_operand    100.00%    100      -      Covered
covered/total bins:      2         2      -
missing/total bins:      0         2      -
% Hit:                   100.00%    100      -
Auto, Default and User Defined Bins:
  bin ADDorUB_max_pos     2         1      -      Covered
  bin ADDorUB_max_neg     1         1      -      Covered

```



Waveform:



Question 2:

Testbench:

```

Assignments > A3 > Q2 > counter_tb.sv > counter_tb > check_rst
1  import counter_pkg::*;
2
3  module counter_tb;
4
5  bit clk, rst_n, load_n, up_down, ce;
6  bit [WIDTH-1:0] data_load;
7  logic max_count, max_count_exp, zero, zero_exp;
8  logic [WIDTH-1:0] count_out, count_out_exp;
9
10 counter_class Obj = new;
11
12 int error_count;
13 int correct_count;
14
15 counter #(.WIDTH(WIDTH)) DUT(
16     .clk(clk),
17     .rst_n(rst_n),
18     .load_n(load_n),
19     .up_down(up_down),
20     .ce(ce),
21     .data_load(data_load),
22     .count_out(count_out),
23     .max_count(max_count),
24     .zero(zero));
25
26 initial begin
27     clk = 0;
28     forever begin
29         #1 clk = ~clk;
30         Obj.clk = clk;
31     end
32 end
33
34 initial begin
35     // COUNTER_6
36     assert_reset;
37     // COUNTER_7
38     repeat (500) begin
39         assert(Obj.randomize());
40         rst_n = Obj.rst_n;
41         load_n = Obj.load_n;
42         up_down = Obj.up_down;
43         ce = Obj.ce;
44         data_load = Obj.data_load;
45
46         check_result(Obj);
47     end
48
49     $display("%t: At end of test error count is %0d and correct count = %0d",
50         $time, error_count, correct_count);
51     $stop;
52 end

```



```

54 task check_result(input counter_class Obj_check);
55     golden_model(Obj_check);
56
57     if (Obj_check.rst_n != 0) begin
58         @(negedge clk);
59         if ((max_count_exp != max_count) ||
60             (zero_exp != zero) ||
61             (count_out_exp != count_out)) begin
62
63             error_count = error_count + 1;
64
65             $display("%t: Error: For rst_n=%0d, load_n=%0d, up_down=%0d, ce=%0d,
66                 data_load=%0d, count_out=%0d",
67                 $time, rst_n, load_n, up_down, ce, data_load, count_out);
68         end
69     else
70         correct_count = correct_count + 1;
71     end
72 else begin
73     check_rst;
74 end
75 endtask

```

```

77 // Golden Model
78 task golden_model(input counter_class Obj_golden);
79     if (!Obj_golden.rst_n)
80         count_out_exp = 0;
81     else if (!Obj_golden.load_n)
82         count_out_exp = Obj_golden.data_load;
83     else if (Obj_golden.ce) begin
84         if (Obj_golden.up_down)
85             count_out_exp = count_out_exp + 1;
86         else
87             count_out_exp = count_out_exp - 1;
88     end
89
90     max_count_exp = (count_out_exp == {(WIDTH){1'b1}}) ? 1 : 0;
91     zero_exp      = (count_out_exp == 0) ? 1 : 0;
92 endtask
93
94 task assert_reset;
95     rst_n = 0;
96     check_rst;
97
98     rst_n = 1;
99 endtask

```

```

101 task check_rst;
102     @(negedge clk);
103     if (count_out != 0 && max_count != 0 && zero == 1) begin
104         error_count = error_count + 1;
105         $display("%t: Error: Reset value is asserted, outputs are with wrong reset values",
106                 $time);
107     end
108     else
109         correct_count = correct_count + 1;
110 endtask
111
112 endmodule

```



Package:

```

Assignments > A3 > Q2 > counter_pkg.sv > ...
1  package counter_pkg;
2  parameter WIDTH = 4;
3
4  class counter_class;
5      rand bit rst_n, load_n, up_down, ce;
6      rand bit [WIDTH-1:0] data_load, count_out;
7      parameter max_count = 2**WIDTH - 1;
8
9      bit clk;
10
11     // COUNTER_0
12     constraint rst_constraint {
13         rst_n dist {0 := 5, 1 := 95};
14     }
15
16     constraint load_constraint {
17         load_n dist {0 := 70, 1 := 30};
18     }
19
20     constraint ce_constraint {
21         ce dist {1 := 70, 0 := 30};
22     }
23
24     covergroup cover_counter @(posedge clk);
25
26         // COUNTER_1
27         cp1 : coverpoint data_load iff(!load_n && rst_n);
28         // COUNTER_2
29         cp2 : coverpoint count_out iff(rst_n && ce && up_down){
30             bins count_out_bins_up[] = {[0 : max_count]};
31         }
32         // COUNTER_3
33         cp3 : coverpoint count_out iff(rst_n && ce && up_down){
34             bins overflow_bin = (max_count ==> 0);
35         }
36         // COUNTER_4
37         cp4 : coverpoint count_out iff(rst_n && ce && !up_down){
38             bins count_out_bins_down[] = {[0 : max_count]};
39         }
40         // COUNTER_5
41         cp5 : coverpoint count_out iff(rst_n && ce && !up_down){
42             bins underflow_bin = (0 ==> max_count);
43         }
44     endgroup
45
46     function new();
47         cover_counter = new();
48     endfunction
49
50 endclass
51 endpackage

```



Design:

```

Assignments > A3 > Q2 > counter.v > counter
1  module counter (clk, rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
2  parameter WIDTH = 4;
3  input clk;
4  input rst_n;
5  input load_n;
6  input up_down;
7  input ce;
8  input [WIDTH-1:0] data_load;
9  output reg [WIDTH-1:0] count_out;
10 output max_count;
11 output zero;
12
13 always @(posedge clk) begin
14     if (!rst_n)
15         count_out <= 0;
16     else if (!load_n)
17         count_out <= data_load;
18     else if (ce)
19         if (up_down)
20             count_out <= count_out + 1;
21         else
22             count_out <= count_out - 1;
23 end
24
25 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
26 assign zero = (count_out == 0)? 1:0;
27
28 endmodule

```

Do File:

```

Assignments > A3 > Q3 > run.do
1  vlib work
2  vlog ALSU_pkg.sv ALSU_GM.v ALSU.v ALSU_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.ALSU_tb -cover
4  add wave *
5  coverage save ALSU_tb.ucdb -onexit
6  run -all
7  coverage exclude -du ALSU -togglenode {cin_reg[1]}
8

```



Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_0	rst_n is deasserted most of the time. load_n is asserted 70% of the time. ce is active 70% of the time.	Randomization under constraints: rst_n dist {0:5, 1:95}, load_n dist {0:70, 1:30}, ce dist {1:70, 0:30}.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_1	When load is asserted and reset is inactive, count_out should take the value of data_load.	Randomization under constraints driving load_n=0 and rst_n=1.	cp1: coverpoint on data_load sampled iff(!load_n && rst_n) — covers all 16 possible load values.	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_2	When reset and load are inactive, count_out increments each cycle while ce=1 and up_down=1.	Randomization for up_down and ce, with ce active 70% of the time.	cp2: coverpoint on count_out iff(rst_n && ce && up_down) with individual bins for all values 0 to max_count.	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_3	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is low	Randomization for up_down, and randomization for enable to be 70% of the time	cp3: transition bin (max_count => 0) on count_out iff(rst_n && ce && up_down).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_4	When reset and load are inactive, count_out decrements each cycle while ce=1 and up_down=0.	Randomization for up_down and ce, with ce active 70% of the time.	cp4: coverpoint on count_out iff(rst_n && ce && !up_down) with individual bins for all values 0 to max_count.	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_5	When counting down, the counter wraps from 0 back to max_count (underflow).	Randomization for up_down=0 and ce=1, running 500 times.	cp5: transition bin (0 => max_count) on count_out iff(rst_n && ce && !up_down).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_6	When reset is asserted, count_out should be 0, zero flag should be 1, and max_count flag should be 0.	Directed stimulus at start of simulation: rst_n forced to 0 via assert_reset task before any randomization.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_7	All counter operations (reset, load, up-count, down-count) function correctly across 500 random stimulus cycles.	repeat(500) loop calling Obj.randomize() each iteration, driving all DUT inputs from randomized class fields.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.



Code Coverage Report:

```

Statement Coverage:
Enabled Coverage      Bins    Hits    Misses Coverage
-----
Statements           7       7       0    100.00%

=====Statement Details=====
Statement Coverage for instance /counter_tb/DUT --
Line    Item          Count    Source
-----
File counter.v
1      module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
2      parameter WIDTH = 4;
3      input clk;
4      input rst_n;
5      input load_n;
6      input up_down;
7      input ce;
8      input [WIDTH-1:0] data_load;
9      output reg [WIDTH-1:0] count_out;
10     output max_count;
11     output zero;
12
13     1      501    always @(posedge clk) begin
14     1      36      if (rst_n)
15     1      count_out <= 0;
16     1      327    else if (load_n)
17     1      count_out <= data_load;
18     1      else if (ce)
19     1      if (up_down)
20     1      count_out <= count_out + 1;
21     1      else
22     1      count_out <= count_out - 1;
23     1      end
24
25     1      443    assign max_count = (count_out == (WIDTH[1'b1]))? 1:0;
26     1      443    assign zero = (count_out == 0)? 1:0;

```

```

Branch Coverage:
Enabled Coverage      Bins    Hits    Misses Coverage
-----
Branches             10      10       0    100.00%

=====Branch Details=====
Branch Coverage for instance /counter_tb/DUT
Line    Item          Count    Source
-----
File counter.v
-----IF Branch-----
14      1      501    Count coming in to IF
14      1      36      if (rst_n)
16      1      327    else if (load_n)
18      1      102    else if (ce)
18      1      36      All False Count
Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----
19      1      102    Count coming in to IF
19      1      58      if (up_down)
21      1      44      else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
25      1      442    Count coming in to IF
25      1      24      assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
25      2      418    assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
26      1      442    Count coming in to IF
26      1      55      assign zero = (count_out == 0)? 1:0;
26      2      387    assign zero = (count_out == 0)? 1:0;
Branch totals: 2 hits of 2 branches = 100.00%

```

```

Toggle Coverage:
Enabled Coverage      Bins    Hits    Misses Coverage
-----
Toggles              30      30       0    100.00%

=====Toggle Details=====
Toggle Coverage for instance /counter_tb/DUT --
Node    1H->0L    0L->1H    "Coverage"
-----
ce      1          1      100.00
clk     1          1      100.00
count_out[3:0] 1          1      100.00
data_load[0:3] 1          1      100.00
load_n  1          1      100.00
max_count 1          1      100.00
rst_n    1          1      100.00
up_down  1          1      100.00
zero     1          1      100.00

Total Node Count = 15
Toggled Node Count = 15
Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

```



Functional Coverage Report:

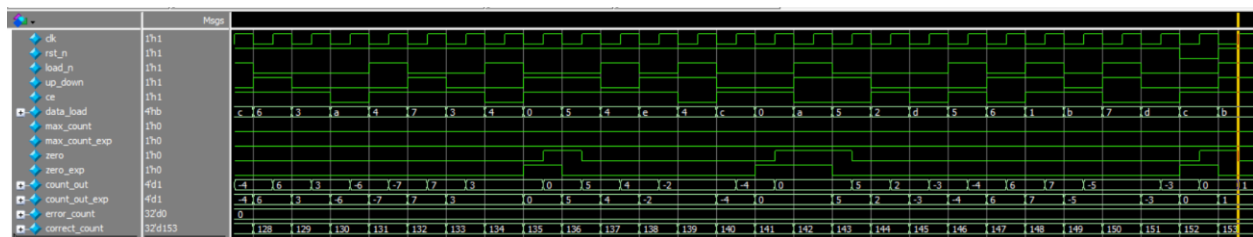
```

=====
=== Instance: /counter_pkg
=== Design Unit: work.counter_pkg
=====

Covergroup Coverage:
  Covergroups          1      na      na    100.00%
  Coverpoints/Crosses  5      na      na      na
  Covergroup Bins      50     50      0    100.00%
  
```

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/ALSU_pkg/ALSU_class		100.00%							
Type g1		100.00%	100	100.00...			auto(0)		
CVP g1::A_cp		100.00%	100	100.00...					
CVP g1::B_cp		100.00%	100	100.00...					
CVP g1::ALU_cp		100.00%	100	100.00...					
INST /ALSU_pkg::ALSU_class::g1		100.00%	100	100.00...					0
CVP A_cp		100.00%	100	100.00...					
bin A_data_0		494	1	100.00...					
bin A_data_max		376	1	100.00...					
bin A_data_min		364	1	100.00...					
bin A_data_walkingones[1]		189	1	100.00...					
bin A_data_walkingones[2]		206	1	100.00...					
default bin A_data_default		1118	-	-					
CVP B_cp		100.00%	100	100.00...					
bin B_data_0		603	1	100.00...					
bin B_data_max		407	1	100.00...					
bin B_data_min		422	1	100.00...					
bin B_data_walkingones[1]		107	1	100.00...					
bin B_data_walkingones[2]		86	1	100.00...					
default bin B_data_default		930	-	-					
CVP ALU_cp		100.00%	100	100.00...					
illegal_bin Bins_invalid		654	-	-					
bin Bins_shift[SHIFT]		390	1	100.00...					
bin Bins_shift[ROTATE]		422	1	100.00...					
bin Bins_arith[ADD]		417	1	100.00...					
bin Bins_arith[MULT]		404	1	100.00...					
bin Bins_bitwise[OR]		421	1	100.00...					
bin Bins_bitwise[XOR]		399	1	100.00...					
bin Bins_trans		1	1	100.00...					

Waveform:



Question 3:

Testbench:

```

Assignments > A3 > Q3 > ALSU_tb.sv > ALSU_tb
1  import ALSU_pkg ::*;
2  module ALSU_tb;
3
4  bit clk, rst;
5  logic cin, serial_in, direction, red_op_A, red_op_B, bypass_A, bypass_B;
6  logic signed [2:0] A, B;
7  opcode_e opcode;
8
9  logic [15:0] leds;
10 logic signed [5:0] out;
11
12 logic [15:0] leds_exp;
13 logic signed [5:0] out_exp;
14
15 localparam FULL_ADDER = "ON";
16 localparam INPUT_PRIORITY = "A";
17
18 integer correct_count, error_count;
19
20 ALSU_class Obj;
21
22 initial begin
23     clk = 0;
24     forever
25         #1 clk = ~clk;
26 end
27
28 ALSU #(.FULL_ADDER(FULL_ADDER), .INPUT_PRIORITY(INPUT_PRIORITY)) DUT(.);
29
30 ALSU_GM #(.FULL_ADDER(FULL_ADDER), .INPUT_PRIORITY(INPUT_PRIORITY)) Golden_Model(
31     .clk(clk),
32     .rst(rst),
33     .cin(cin),
34     .serial_in(serial_in),
35     .direction(direction),
36     .red_op_A(red_op_A),
37     .red_op_B(red_op_B),
38     .bypass_A(bypass_A),
39     .bypass_B(bypass_B),
40     .A(A),
41     .B(B),
42     .opcode(opcode),
43
44     .out(out_exp),
45     .leds(leds_exp)
46 );
47
48 always@(posedge rst)
49     Obj.g1.stop();
50
51 always@(negedge rst)
52     Obj.g1.start();
53
54 always@(posedge clk)
55     Obj.g1.sample();

```



```

57 initial begin
58
59     correct_count = 0;
60     error_count = 0;
61
62     Obj = new();
63
64     //ALSU_5
65     @(negedge clk);
66
67     assert_reset();
68     assert_reset();

```

```

70     //ALSU_6
71     for(int i = 0; i < 320; i++) begin
72
73         Obj.Assignment_2.constraint_mode(1);
74         Obj.Assignment_3.constraint_mode(0);
75
76         assert(Obj.randomize());
77         opcode = Obj.opcode;
78         A = Obj.A;
79         B = Obj.B;
80         rst = Obj.rst;
81         cin = Obj.cin;
82         serial_in = Obj.serial_in;
83         direction = Obj.direction;
84         red_op_A = Obj.red_op_A;
85         red_op_B = Obj.red_op_B;
86         bypass_A = Obj.bypass_A;
87         bypass_B = Obj.bypass_B;
88         check_result();
89
90         Obj.constraint_mode(0);
91
92         rst = 0;
93         bypass_A = 0;
94         bypass_B = 0;
95         red_op_A = 0;
96         red_op_B = 0;

```

```

98         //ALSU_7
99         Obj.Assignment_3.constraint_mode(1);
100         assert(Obj.randomize());
101         for(int j = 0; j < 6; j++) begin
102             opcode = Obj.my_array[j];
103             check_result();
104         end
105
106     end

```



```

107 //ALSU_8
108 for(int i = 0; i < 1000; i++) begin
109     Obj.constraint_mode(0);
110     Obj.Assignment_2.constraint_mode(1);
111     assert(Obj.randomize());
112     opcode = Obj.opcode;
113     A = Obj.A;
114     B = Obj.B;
115     rst = Obj.rst;
116     cin = Obj.cin;
117     serial_in = Obj.serial_in;
118     direction = Obj.direction;
119     red_op_A = Obj.red_op_A;
120     red_op_B = Obj.red_op_B;
121     bypass_A = Obj.bypass_A;
122     bypass_B = Obj.bypass_B;
123     check_result();
124 end
125
126 Obj.opcode = OR;
127 @(posedge clk);
128 //ALSU_9
129 for(opcode_e op = XOR; op < INVALID_6; op = op.next()) begin
130     @(negedge clk);
131     Obj.opcode = op;
132     @(posedge clk);
133 end
134
135 $display("%t: At end of test error count is %0d and correct count = %0d",
136         $time, error_count, correct_count);
137 $stop;
138 end
139
140 task check_result;
141     @(negedge clk);
142     if(leds != leds_exp || out != out_exp)
143     begin
144         error_count++;
145         $display("%t: Error: OPCODE = %0d | leds = %0d | out = %0d | leds_exp = %0d |
146                 out_exp = %0d", $time, opcode, leds, out, leds_exp, out_exp);
147     end
148     else
149         correct_count++;
150 endtask
151
152 task assert_reset;
153     rst = 1;
154     check_result();
155     rst = 0;
156     check_result();
157 endtask
158
159 endmodule

```



Package:

```

17     covergroup g1;
18         //ALSU_0
19         A_cp: coverpoint A {
20             bins A_data_0 = {ZERO};
21             bins A_data_max = {MAXPOS};
22             bins A_data_min = {MAXNEG};
23             bins A_data_default = default;
24             bins A_data_walkingones[] = {3'b001, 3'b010, 3'b100} iff(red_op_A);
25         }
26         //ALSU_1
27         B_cp: coverpoint B {
28             bins B_data_0 = {ZERO};
29             bins B_data_max = {MAXPOS};
30             bins B_data_min = {MAXNEG};
31             bins B_data_default = default;
32             bins B_data_walkingones[] = {3'b001, 3'b010, 3'b100} iff(red_op_B && !red_op_A);
33         }
34         //ALSU_2
35         ALU_cp: coverpoint opcode {
36             bins Bins_shift[] = {SHIFT, ROTATE};
37             bins Bins_arith[] = {ADD, MULT};
38             bins Bins_bitwise[] = {OR, XOR};
39             illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
40             bins Bins_trans = (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE);
41         }
42     endgroup

```

Assignments > A3 > Q3 > ALSU_pkg.sv > {} ALSU_pkg > ALSU_class

```

1  package ALSU_pkg;
2  typedef enum logic [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
3
4  parameter INPUT_PRIORITY = "A";
5  parameter FULL_ADDER = "ON";
6
7  localparam MAXPOS = 3;
8  localparam MAXNEG = -4;
9  localparam ZERO = 0;
10
11 class ALSU_class;
12     rand logic rst, cin, serial_in, direction, red_op_A, red_op_B, bypass_A, bypass_B;
13     rand opcode_e opcode;
14     rand logic signed [2:0] A, B;
15     bit clk;

```



```
44 //ALU_3
45 > constraint Assignment_2 { ...
97 }
98
99 rand opcode_e my_array [6];
100 //ALU_4
101 constraint Assignment_3{
102     unique(my_array);
103     foreach(my_array[i])
104         my_array[i] inside {[OR:ROTATE]};
105 }
106
107 function new();
108 this.rst = 0;
109 this.cin = 0;
110 this.serial_in = 0;
111 this.direction = 0;
112 this.red_op_A = 0;
113 this.red_op_B = 0;
114 this.bypass_A = 0;
115 this.bypass_B = 0;
116 this.opcode = OR;
117 this.A = 0;
118 this.B = 0;
119 g1 = new();
120 endfunction
121 endclass
122 endpackage
```



Design:

```

Assignments > A3 > Q3 > ALSU.v > ALSU
1  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction
2      parameter INPUT_PRIORITY = "A";
3      parameter FULL_ADDER = "ON";
4      input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5      input [2:0] opcode;
6      input signed [2:0] A, B;
7      output reg [15:0] leds;
8      output reg signed [5:0] out;
9
10     reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11     reg signed [1:0] cin_reg;
12     reg [2:0] opcode_reg;
13     reg signed [2:0] A_reg, B_reg;
14
15     wire invalid_red_op, invalid_opcode, invalid;
16
17     assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
18     assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
19     assign invalid = invalid_red_op | invalid_opcode;
20
21     always @(posedge clk or posedge rst) begin
22         if(rst) begin
23             cin_reg <= 0;
24             red_op_B_reg <= 0;
25             red_op_A_reg <= 0;
26             bypass_B_reg <= 0;
27             bypass_A_reg <= 0;
28             direction_reg <= 0;
29             serial_in_reg <= 0;
30             opcode_reg <= 0;
31             A_reg <= 0;
32             B_reg <= 0;
33         end else begin
34             cin_reg <= cin;
35             red_op_B_reg <= red_op_B;
36             red_op_A_reg <= red_op_A;
37             bypass_B_reg <= bypass_B;
38             bypass_A_reg <= bypass_A;
39             direction_reg <= direction;
40             serial_in_reg <= serial_in;
41             opcode_reg <= opcode;
42             A_reg <= A;
43             B_reg <= B;
44         end
45     end
46
47     always @(posedge clk or posedge rst) begin
48         if(rst) begin
49             leds <= 0;
50         end else begin
51             if (invalid)
52                 leds <= ~leds;
53             else
54                 leds <= 0;
55         end
56     end

```



```

58   always @(posedge clk or posedge rst) begin
59       if(rst) begin
60           out <= 0;
61       end
62       else begin
63           if (bypass_A_reg && bypass_B_reg)
64               out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
65           else if (bypass_A_reg)
66               out <= A_reg;
67           else if (bypass_B_reg)
68               out <= B_reg;
69           else if (invalid)
70               out <= 0;
71           else begin
72               case (opcode_reg)
73                   3'h0: begin
74                       if (red_op_A_reg && red_op_B_reg)
75                           out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
76                       else if (red_op_A_reg)
77                           out <= |A_reg;
78                       else if (red_op_B_reg)
79                           out <= |B_reg;
80                       else
81                           out <= A_reg | B_reg;
82                   end
83                   3'h1: begin
84                       if (red_op_A_reg && red_op_B_reg)
85                           out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
86                       else if (red_op_A_reg)
87                           out <= ^A_reg;
88                       else if (red_op_B_reg)
89                           out <= ^B_reg;
90                       else
91                           out <= A_reg ^ B_reg;
92                   end
93                   3'h2: out <= A_reg + B_reg + cin_reg;
94                   3'h3: out <= A_reg * B_reg;
95                   3'h4: begin
96                       if (direction_reg)
97                           out <= {out[4:0], serial_in_reg};
98                       else
99                           out <= {serial_in_reg, out[5:1]};
100                   end
101                   3'h5: begin
102                       if (direction_reg)
103                           out <= {out[4:0], out[5]};
104                       else
105                           out <= {out[0], out[5:1]};
106                   end
107               endcase
108           end
109       end
110   end
111 endmodule

```



Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_0	Input A must cover zero, max positive, max negative, and walking-ones patterns when reduction op on A is active.	Randomization under Assignment_2 constraints A toward MAXPOS, MAXNEG, ZERO and walking-ones patterns when red_op_A=1.	A_cp: bins for ZERO, MAXPOS, MAXNEG, default, and walking-ones (3'b001, 3'b010, 3'b100) iff(red_op_A).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_1	Input B must cover zero, max positive, max negative, and walking-ones patterns when reduction op on B is active and A reduction is inactive.	Randomization under Assignment_2 constraints B toward walking-ones patterns when red_op_B=1 and red_op_A=0.	B_cp: bins for ZERO, MAXPOS, MAXNEG, default, and walking-ones (3'b001, 3'b010, 3'b100) iff(red_op_B && !red_op_A).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_2	Opcode must cover all valid operations, the transitions in order, and never take invalid values.	Randomization under Assignment_2 with weighted dist across all opcodes including INVALID_6/7 at lower weight.	ALU_cp: bins for shift {SHIFT,ROTATE}, arith {ADD,MULT}, bitwise {OR,XOR}, illegal_bins for {INVALID_6,INVALID_7}, transition bin (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_3	All inputs are randomized with weighted distributions: opcodes equally weighted, bypass and rst favoring deasserted, A/B biased toward boundary values for arithmetic, walking-ones for bitwise, and direction biased for shift/rotate.	Assignment_2 constraint drives dist on opcode, bypass_A/B, rst, A, B, red_op_A/B, direction conditioned on opcode group.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_4	The ALSU must produce correct outputs for all 6 valid opcodes applied in a unique, non-repeating sequence.	Assignment_3 constraint forces my_array to hold all 6 opcodes (OR to ROTATE) uniquely using unique(my_array) and foreach range constraint.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_5	On reset assertion, outputs (out and leds) must be low.	Directed stimulus: rst forced to 1 then 0 twice using assert_reset task before any randomization begins.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_6	All ALSU operations produce correct outputs under fully randomized constrained stimulus across 320 iterations.	320-iteration loop with Assignment_2 constraints active and Assignment_3 constraints disabled. All inputs randomized and driven to DUT each iteration.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_7	All 6 valid opcodes exercised in a unique sequential order within each outer iteration, with bypass and reduction signals deasserted.	Assignment_3 constraints enabled after disabling all constraints. randomize() called once to fill my_array with 6 unique opcodes. rst, bypass, red_op signals forced to 0.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_8	All ALSU operations produce correct outputs under an extended 1000-iteration constrained random stimulus for deeper corner-case coverage.	1000-iteration loop with all constraints disabled except Assignment_2 constraints.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_9	The opcode transition coverage bin (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE) must be explicitly hit in the exact required order.	Obj.opcode set to OR then goes through XOR to ROTATE.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.



Do File:

```

Assignments > A3 > Q3 > ≡ run.do
1  vlib work
2  vlog ALSU_pkg.sv ALSU_GM.v ALSU.v ALSU_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.ALSU_tb -cover
4  add wave *
5  coverage save ALSU_tb.ucdb -onexit
6  run -all
7  coverage exclude -du ALSU -togglenode {cin_reg[1]}

```

Code Coverage Report:

```

=====
Branch Coverage:

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	32	32	0	100.00%

```

Statement Coverage:

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	48	48	0	100.00%

```

Toggle Coverage:

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	118	118	0	100.00%



Functional Coverage Report:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/ALSU_pkg/ALSU_class		100.00%				
TYPE g1		100.00%	100	100.00...		✓
CVP g1::A_cp		100.00%	100	100.00...		✓
CVP g1::B_cp		100.00%	100	100.00...		✓
CVP g1::ALU_cp		100.00%	100	100.00...		✓
INST \ALSU_pkg::ALSU_class::g1		100.00%	100	100.00...		✓
CVP A_cp		100.00%	100	100.00...		✓
CVP B_cp		100.00%	100	100.00...		✓
CVP ALU_cp		100.00%	100	100.00...		✓

```
=====
=== Instance: /ALSU_pkg
=== Design Unit: work.ALSU_pkg
=====
```

Covergroup Coverage:					
Covergroups	1	na	na	100.00%	
Coverpoints/Crosses	3	na	na	na	
Covergroup Bins	17	17	0	100.00%	

```
-----
```

Covergroup	Metric	Goal	Bins	Status

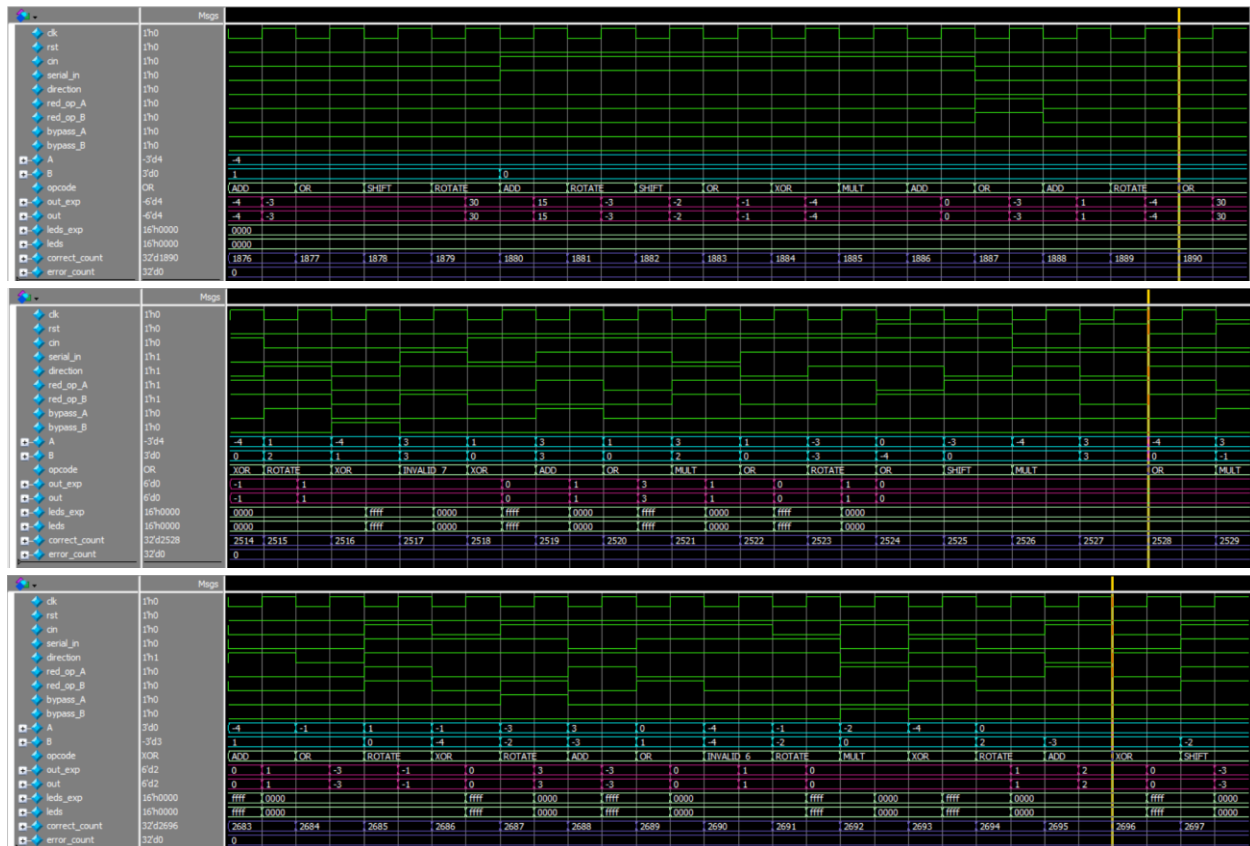
TYPE /ALSU_pkg/ALSU_class/g1	100.00%	100	-	Covered
covered/total bins:	17	17	-	
missing/total bins:	0	17	-	
% Hit:	100.00%	100	-	
Coverpoint A_cp	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Coverpoint B_cp	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Coverpoint ALU_cp	100.00%	100	-	Covered
covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
Covergroup instance \ALSU_pkg::ALSU_class::g1	100.00%	100	-	Covered
covered/total bins:	17	17	-	
missing/total bins:	0	17	-	
% Hit:	100.00%	100	-	
Coverpoint A_cp	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	



Coverpoint B_cp	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Coverpoint ALU_cp	100.00%	100	-	Covered
covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
Covergroup instance \\\ALSU_pkg::ALSU_class::g1	100.00%	100	-	Covered
covered/total bins:	17	17	-	
missing/total bins:	0	17	-	
% Hit:	100.00%	100	-	
Coverpoint A_cp	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
bin A_data_0	494	1	-	Covered
bin A_data_max	376	1	-	Covered
bin A_data_min	364	1	-	Covered
bin A_data_walkingones[1]	189	1	-	Covered
bin A_data_walkingones[2]	206	1	-	Covered
default bin A_data_default	1118		-	Occurred
Coverpoint B_cp	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
bin B_data_0	603	1	-	Covered
bin B_data_max	407	1	-	Covered
bin B_data_min	422	1	-	Covered
bin B_data_walkingones[1]	107	1	-	Covered
bin B_data_walkingones[2]	86	1	-	Covered
default bin B_data_default	930		-	Occurred
Coverpoint ALU_cp	100.00%	100	-	Covered
covered/total bins:	7	7	-	
missing/total bins:	0	7	-	
% Hit:	100.00%	100	-	
illegal_bin Bins_invalid	654		-	Occurred
bin Bins_shift[SHIFT]	390	1	-	Covered
bin Bins_shift[ROTATE]	422	1	-	Covered
bin Bins_arith[ADD]	417	1	-	Covered
bin Bins_arith[MULT]	404	1	-	Covered
bin Bins_bitwise[OR]	421	1	-	Covered
bin Bins_bitwise[XOR]	399	1	-	Covered
bin Bins_trans	1	1	-	Covered



Waveform:



Question 4:

Testbench:

Assignments > A3 > Q4 > mem_tb.sv > mem_tb > check9Bits

```

1  module mem_tb;
2
3      bit clk;
4      logic write, read;
5      logic [7:0] data_in,data_out;
6      logic [15:0] address;
7
8      localparam TESTS = 100;
9
10     logic [15:0] address_array[];
11     logic [7:0] data_to_write_array[];
12     logic [7:0] data_read_expect_assoc[int];
13     logic [7:0] data_read_queue[$];
14
15     initial begin
16         clk = 0;
17         forever #2 clk = ~clk;
18     end
19
20     my_mem DUT (
21         .clk(clk),
22         .write(write),
23         .read(read),
24         .data_in(data_in),
25         .address(address),
26         .data_out(data_out)
27     );
28
29     integer error_count, correct_count;
30
31     task stimulus_gen;
32         address_array = new[TESTS];
33         data_to_write_array = new[TESTS];
34         for(int i = 0; i < TESTS; i++)begin
35             address_array[i] = $urandom;
36             data_to_write_array[i] = $random;
37         end
38     endtask
39
40     task golden_model;
41         for(int j = 0; j < TESTS; j++)begin
42             data_read_expect_assoc[address_array[j]] = data_to_write_array[j];
43         end
44     endtask
45
46     task check9Bits;
47
48         @(negedge clk);
49
50         if (data_out == data_read_expect_assoc[address]) begin
51             correct_count++;
52         end else begin
53             error_count++;
54             $display("ERROR: Address = 0x%0h | Expected = %0d | Result = %0d",
55                     address, data_read_expect_assoc[address], data_out);
56         end
57     endtask

```




```

59     initial begin
60
61         write      = 0;
62         read       = 0;
63         data_in    = 0;
64         address    = 0;
65         error_count = 0;
66         correct_count = 0;
67
68         repeat(2)@(negedge clk);
69         //MEM_1
70         stimulus_gen();
71         golden_model();
72
73         //MEM_2
74         write = 1;
75         for(int k = 0; k < TESTS; k++)begin
76             @(negedge clk);
77             address = address_array[k];
78             data_in = data_to_write_array [k];
79         end

```

```

81         repeat(2)@(negedge clk);
82
83         //MEM3
84         write      = 0;
85         read       = 1;
86
87         for(int l = 0; l < TESTS; l++)begin
88             address = address_array[l];
89             check9Bits();
90             data_read_queue.push_back(data_out);
91         end
92
93         //MEM_4
94         write      = 0;
95         read       = 0;
96         repeat(2)@(negedge clk);
97
98         while (data_read_queue.size() > 0)
99             $display("data = %0d", data_read_queue.pop_front());
100
101         $display("%t: At end of test error count is %0d and correct count = %0d",
102             $time, error_count, correct_count);
103     $stop();
104     end
105
106 endmodule

```



Design:

```

Assignments > A3 > Q4 > my_mem.sv > my_mem
1  module my_mem(
2      input clk,
3      input write,
4      input read,
5      input [7:0] data_in,
6      input [15:0] address,
7      output reg [7:0] data_out
8  );
9
10     logic [8:0] mem_array [int];
11
12     always @(posedge clk) begin
13         if (write)
14             mem_array[address] = {~^data_in, data_in};
15         else if (read)
16             data_out = mem_array[address];
17     end
18 endmodule

```

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
MEM_1	On write=1 at posedge clk, RAM stores data_in at the given address. Write has higher priority than read.	100 random addresses and data values generated. write=1, read=0.	-	A checker in the testbench to make sure the output is correct
MEM_2	On read=1 at posedge clk (and write=0), RAM outputs the stored data at the given address on data_out.	After all writes complete, same address_array is iterated with read=1, write=0.	-	A checker in the testbench to make sure the output is correct
MEM_3	Even parity is calculated on data written to RAM and stored in the 9th bit of the memory array.	Random 8-bit data written using data_to_write_array.	-	A checker in the testbench to make sure the output is correct
MEM_4	All written data can be correctly retrieved from the associative memory across 100 random addresses.	Dynamic arrays address_array and data_to_write_array of size TESTS=100 have all stimulus.	-	A checker in the testbench to make sure the output is correct

Do File:

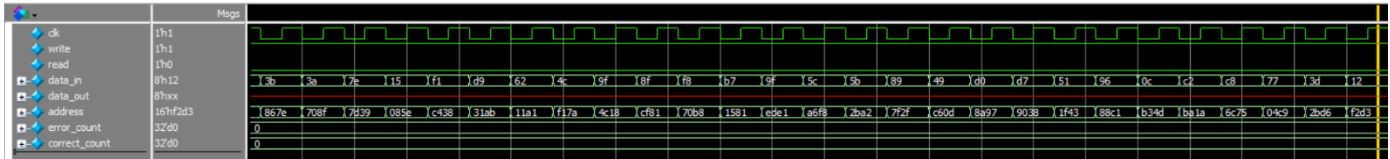
```

Assignments > A3 > Q4 > run.do
1  vlib work
2  vlog my_mem.sv mem_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.mem_tb -cover
4  add wave *
5  coverage save mem_tb.ucdb -onexit
6  run -all
7

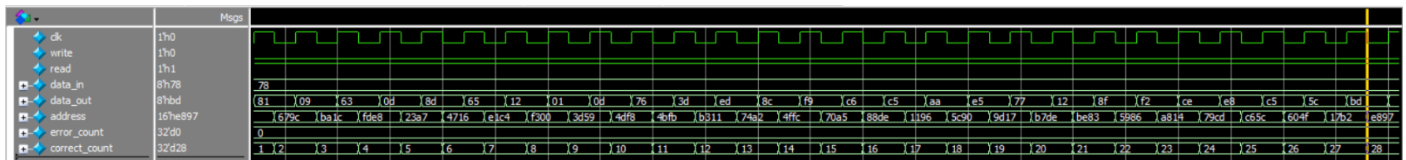
```



Waveform:



Write Operations



Read Operations

